U24AI006

ASSIGNMENT -11

1. Consider the following districts in the map of Gujarat.

Color this map with the lowest number of colors posing it as a constraint satisfaction problem so that no two adjacent districts are colored the same. [Diu is not a district of Gujarat and Rann of Kuchchh and Little Rann of Kuchchh are also not districts]

2. Solve the following cryptarithmetic problem posing it as a constraint satisfaction problem. Each letter represents a unique digit from 0 to 9. The leading letters (S and M) cannot be 0. Solve:

```
    SEND
+   MORE
_________
   MONEY
```

U24AI006

CODE(1):

```
raw_graph = {
    "Kachchh": ["Surendranagar", "Patan", "Jamnagar"],
    "Banaskantha": ["Patan", "Mehsana", "Sabarkantha"],
    "Patan": ["Banaskantha", "Mehsana", "Surendranagar", "Kachchh"],
    "Mehsana": ["Banaskantha", "Patan", "Sabarkantha", "Gandhinagar", "Ahmedabad"],
    "Sabarkantha": ["Banaskantha", "Mehsana", "Gandhinagar", "Kheda", "Panchmahal"],
    "Gandhinagar": ["Mehsana", "Sabarkantha", "Ahmedabad", "Kheda"],
    "Kheda": ["Sabarkantha", "Gandhinagar", "Ahmedabad", "Anand", "Vadodara",
"Panchmahal"],
    "Panchmahal": ["Sabarkantha", "Kheda", "Vadodara", "Dahod"],
    "Dahod": ["Panchmahal", "Vadodara"],
    "Ahmedabad": ["Mehsana", "Gandhinagar", "Kheda", "Anand", "Surendranagar"],
    "Anand": ["Ahmedabad", "Kheda", "Vadodara", "Bharuch"],
    "Vadodara": ["Kheda", "Anand", "Panchmahal", "Dahod", "Narmada", "Bharuch"],
    "Narmada": ["Vadodara", "Bharuch", "Surat", "Dangs"],
    "Bharuch": ["Vadodara", "Anand", "Narmada", "Surat"],
    "Surat": ["Bharuch", "Narmada", "Navsari", "Dangs"],
    "Navsari": ["Surat", "Valsad", "Dangs"],
    "Valsad": ["Navsari", "Dangs"],
    "Dangs": ["Narmada", "Surat", "Navsari", "Valsad"],
    "Surendranagar": ["Kachchh", "Patan", "Ahmedabad", "Rajkot", "Bhavnagar",
"Jamnagar"],
    "Jamnagar": ["Kachchh", "Rajkot", "Porbandar", "Surendranagar"],
    "Rajkot": ["Surendranagar", "Jamnagar", "Porbandar", "Junagadh", "Amreli"],
    "Porbandar": ["Jamnagar", "Rajkot", "Junagadh"],
    "Junagadh": ["Porbandar", "Rajkot", "Amreli"],
    "Amreli": ["Rajkot", "Junagadh", "Bhavnagar"],
    "Bhavnagar": ["Surendranagar", "Amreli", "Anand", "Vadodara"]
}


def make_graph(data):
    graph = {}
    for district, neighbors in data.items():
        graph.setdefault(district, set()).update(neighbors)
        for neighbor in neighbors:
            graph.setdefault(neighbor, set()).add(district)
    return {district: sorted(neighbors) for district, neighbors in graph.items()}


def is_safe(district, color, assignment, graph):
    for neighbor in graph[district]:
        if assignment.get(neighbor) == color:
            return False
```

```
    return True


def backtrack(index, order, colors, assignment, graph):
    if index == len(order):
        return True

    district = order[index]

    for color in colors:
        if is_safe(district, color, assignment, graph):
            assignment[district] = color
            if backtrack(index + 1, order, colors, assignment, graph):
                return True
            assignment.pop(district)

    return False


def solve_map(graph):
    color_names = ["Red", "Green", "Blue", "Yellow"]
    order = sorted(graph, key=lambda district: len(graph[district]), reverse=True)

    for count in range(1, len(color_names) + 1):
        assignment = {}
        if backtrack(0, order, color_names[:count], assignment, graph):
            return count, assignment

    return None, None


def main():
    graph = make_graph(raw_graph)
    min_colors, assignment = solve_map(graph)

    if assignment is None:
        print("No solution found.")
        return

    print("Gujarat map coloring")
    print("Minimum colors needed:", min_colors)
    print()
    print("District colors:")

    for district in raw_graph:
```

```
        print(district + ": " + assignment[district])



if __name__ == "__main__":
    main()
```

OUTPUT:

```
Gujarat map coloring
Minimum colors needed: 3

District colors:
Kachchh: Blue
Banaskantha: Green
Patan: Red
Mehsana: Blue
Sabarkantha: Red
Gandhinagar: Green
Kheda: Blue
Panchmahal: Green
Dahod: Blue
Ahmedabad: Red
Anand: Green
Vadodara: Red
Narmada: Green
Bharuch: Blue
Surat: Red
Navsari: Green
Valsad: Red
Dangs: Blue
Surendranagar: Green
Jamnagar: Red
Rajkot: Blue
Porbandar: Green
Junagadh: Red
Amreli: Green
Bhavnagar: Blue
```

CODE(2):

```
letters = ["S", "E", "N", "D", "M", "O", "R", "Y"]


def choices(letter, assignment, used):
    if letter in assignment:
        return [assignment[letter]]
    if letter in ("S", "M"):
        return [digit for digit in range(1, 10) if digit not in used]
    return [digit for digit in range(10) if digit not in used]


def put(letter, digit, assignment, used):
    if letter in assignment:
        return assignment[letter] == digit, False
    if digit in used:
        return False, False
    if letter in ("S", "M") and digit == 0:
        return False, False
    assignment[letter] = digit
    used.add(digit)
    return True, True


def remove(letter, digit, added, assignment, used):
    if added:
        del assignment[letter]
        used.remove(digit)


def solve_thousands(carry, assignment, used):
    for s in choices("S", assignment, used):
        ok_s, added_s = put("S", s, assignment, used)
        if not ok_s:
            continue

        for m in choices("M", assignment, used):
            ok_m, added_m = put("M", m, assignment, used)
            if ok_m:
                total = assignment["S"] + assignment["M"] + carry
                if total % 10 == assignment["O"] and total // 10 == assignment["M"]:
                    return True
            remove("M", m, added_m, assignment, used)

        remove("S", s, added_s, assignment, used)
```

```
    return False


def solve_hundreds(carry, assignment, used):
    for o in choices("O", assignment, used):
        ok_o, added_o = put("O", o, assignment, used)
        if not ok_o:
            continue

        total = assignment["E"] + assignment["O"] + carry
        if total % 10 == assignment["N"]:
            next_carry = total // 10
            if solve_thousands(next_carry, assignment, used):
                return True

        remove("O", o, added_o, assignment, used)

    return False


def solve_tens(carry, assignment, used):
    for n in choices("N", assignment, used):
        ok_n, added_n = put("N", n, assignment, used)
        if not ok_n:
            continue

        for r in choices("R", assignment, used):
            ok_r, added_r = put("R", r, assignment, used)
            if ok_r:
                total = assignment["N"] + assignment["R"] + carry
                if total % 10 == assignment["E"]:
                    next_carry = total // 10
                    if solve_hundreds(next_carry, assignment, used):
                        return True
            remove("R", r, added_r, assignment, used)

        remove("N", n, added_n, assignment, used)

    return False


def solve_units(assignment, used):
    for d in choices("D", assignment, used):
        ok_d, added_d = put("D", d, assignment, used)
```

```
        if not ok_d:
            continue

        for e in choices("E", assignment, used):
            ok_e, added_e = put("E", e, assignment, used)
            if ok_e:
                total = assignment["D"] + assignment["E"]
                y = total % 10
                carry = total // 10
                ok_y, added_y = put("Y", y, assignment, used)
                if ok_y and solve_tens(carry, assignment, used):
                    return True
                remove("Y", y, added_y, assignment, used)
            remove("E", e, added_e, assignment, used)

        remove("D", d, added_d, assignment, used)

    return False


def word_value(word, assignment):
    number = ""
    for letter in word:
        number += str(assignment[letter])
    return int(number)


def main():
    assignment = {}
    used = set()

    if not solve_units(assignment, used):
        print("No solution found.")
        return

    send = word_value("SEND", assignment)
    more = word_value("MORE", assignment)
    money = word_value("MONEY", assignment)

    print("Solution found")
    print("SEND =", send)
    print("MORE =", more)
    print("MONEY =", money)
    print("Letter values:")
```

```
    for letter in letters:
        print(f"{letter} = {assignment[letter]}")


if __name__ == "__main__":
    main()
```

OUTPUT:

```
Solution found
SEND = 9567
MORE = 1085
MONEY = 10652
Letter values:
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2
```